

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	Fixed (→ Fixed.md)	5
Bug	Operator-blocked	1
Bug	Queued	15
Task	In progress	3
Task	Operator-blocked	3
Task	Queued	33
Task	Ready for testing	2
TOTAL		62

Items

ATM ID	Type	Status	Severity	Description
BOB-008	Bug	Operator-blocked	—	RuTracker automated login blocked by CAPTCHA
BOB-065	Task	Queued	High	Lava P2: Egress diagnosis and VPN-hos SOCKS routing (containers pkg/egress)
BOB-066	Task	In progress	High	Lava P3: BOBA_UPSTREAM_PROXY in download-proxy + qBitTorrent-go + Jac + compose env-forward
BOB-068	Bug	Queued	High	RD2-00: unattributed, unreviewed Auto commit mechanism pushing to main
BOB-069	Task	Queued	High	RD2-40: §11.4.238 QA-discovery-channel ledger — ongoing coverage-escape back
BOB-070	Bug	Queued	High	RD2-41: pre-build mutation-marker sca carrier false-positive silently defeats en pre-build gate
BOB-074	Task	Queued	Medium	RD2-07: DDoS-class testing fully absent from the mandated test-type matrix

ATM ID	Type	Status	Severity	Description
BOB-077	Task	Operator-blocked	High	RD2-10: Identify second host running the Auto-commit rsync/sync mechanism (OPERATOR-DECISION)
BOB-078	Task	Queued	High	RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit/push script OR retire it
BOB-079	Task	Queued	Medium	RD2-12: Retroactive attributed history notes for GA-18/21/22/25/26/27 changes (never rewrite published history)
BOB-080	Task	Queued	High	RD2-13: Retroactive Fable-xhigh code review of the substantive Auto-commit
BOB-081	Task	Queued	High	RD2-14: Author CONTINUATION.md Session 15 entry (currently 53 days / 24 commits behind HEAD)
BOB-082	Task	Queued	High	RD2-15: Create BOB-064..067 workable items for the four Lava-porting findings (closes GA-05)
BOB-083	Task	Queued	Medium	RD2-16: Regenerate browser_extension features/codegraph Status.md + Summary HTML/PDF siblings
BOB-084	Task	Queued	High	RD2-17: Reconcile BOB-008 DB/MD bo drift via the workable-items tool
BOB-085	Task	Queued	Medium	RD2-18: Create top-level Boba proxy/merge-service v1.0.0 readiness ledger (closes GA-10)
BOB-086	Task	Queued	Medium	RD2-19: Fix BOB-009/BOB-010 evidence_path + backfill item_history for 56 silent closures
BOB-087	Task	Queued	High	RD2-20: Wire docs_chain / commit-search sync hook per §11.4.106(F) so DB writes cannot land without MD mirror
BOB-088	Task	Queued	Medium	RD2-21: Complete/verify README Tracked-Items + Status Documents tab row-completeness (GA-07 remainder)
BOB-089	Task	Queued	High	RD2-24: RED-first tests for start.sh reload_python/reload_plugins/recreate_stack (closes test-half of GA-2)
BOB-090	Task	Queued	Medium	RD2-25: HelixQA Challenge entry exercising all three start.sh subcommands end-to-end against real compose stack
BOB-092	Bug	Queued	Medium	RD2-27: Remove test_live_stack_evidence.py:265 nnmclud SKIP-on-404 fallback + verify live 200 (closes GA-13)
BOB-093	Task	Queued	High	

ATM ID	Type	Status	Severity	Description
				RD2-28: Live compose bring-up + verify rutracker ReDoS regex bounds deployed container (closes runtime half of GA-12)
BOB-094	Task	Queued	Medium	RD2-29: Author tests/stress/ test_tracker_fetch_stress_chaos.py with §11.4.85 fault injection
BOB-095	Task	Queued	Medium	RD2-30: Author tests/stress/ test_scheduler_hooks_sse_stress_chaos for Go-side triangle
BOB-096	Task	Queued	Medium	RD2-31: Extend qBitTorrent-go jackett_db_test.go with real process-kill resource-exhaustion fault injection
BOB-097	Task	Queued	Low	RD2-32: Author DDoS-class coverage for exposed download-proxy/merge endpoint (canonical impl of RD2-07)
BOB-098	Task	Queued	Medium	RD2-34: Parametrize 20 hardcoded / Volumes/T7 paths in helixqa banks with PROJECT_ROOT (closes GA-23)
BOB-099	Bug	Queued	Medium	RD2-36: Fix guard-forbidden-command substring-match false-positive class + a const033-poweroff-signal-triage carrier EXCLUDE_PATHS
BOB-100	Task	Queued	Low	RD2-39: Bump submodules/jackett one commit (canonical impl of RD2-09)
BOB-101	Task	Operator- blocked	High	GA-19/RW-09: Is -profile go parity still a release goal? (gates RW-10..13) — OPERATOR-DECISION
BOB-102	Task	Operator- blocked	Medium	RW-05: LAN-exposure threat model — k tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION
BOB-104	Task	In progress	Medium	§11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge
BOB-105	Task	Queued	Medium	§11.4.238 followup: mechanical §11.4.227(B) anchor-block-integrity che
BOB-106	Task	Queued	Medium	§11.4.238 followup: §11.4.84 quiescenc check helper for the unattributed auto- commit path
BOB-107	Task	Queued	Medium	§11.4.238 followup: pre-dispatch existe check for subagent task-brief source in
BOB-109	Task	Queued	Medium	BOB-074 followup: scaling-class test coverage absent from mandated test-ty matrix
BOB-110	Task	Queued	Medium	BOB-074 followup: UX-class test covera (accessibility/usability) absent
BOB-111	Task		High	

ATM ID	Type	Status	Severity	Description
		In progress		BOB-074 followup: configure real rate limiting for boba's 3 public HTTP endpoints
BOB-113	Task	Queued	Low	BOB-074 followup: add wrk to dev tooling for DDoS/load challenges
BOB-114	Task	Queued	Medium	BOB-074 followup: self-validation golden bad fixture for the rate-limit detector
BOB-118	Bug	Queued	High	README.md python-tests badge claims 585 passing; pytest -collect-only measures 5235 (9x stale/wrong)
BOB-120	Bug	Queued	Critical	3rd forced-logout incident 2026-08-18 23:45:49 — SIGKILL user@1000 + preventive-timer-inside-user-slice architectural gap
BOB-121	Task	Ready for testing	Important	External watchdog for the forced-logout architectural gap (task #85, incident #105) Potential production slowapi/starlette defect flagged by Task 105 subagent — CLOSED 2026-08-21. CONFIRMED as a real defect but ALREADY REMEDIATED by commit 44f3bbe — and this ticket's PREMISE is REFUTED. It is not a slowapi/starlette incompatibility: the mechanism reproduces IDENTICALLY at host starlette 1.4.1 and container 1.6.0, and slowapi's own docs state the contract verbatim (docs/index.md, fetched 2026-08-21). The true root cause is a documented API-user contract — headers_enabled=True requires a response: Response parameter when the endpoint does not itself return a Response. The REAL remaining work was that NO regression guard existed: the code prior references were comments in tests that bypass the decorator via wrapped endpoints never execute slowapi's wrapper — a fix for the defect with nothing falsifiable guarding against it. Added tests/unit/test_bob129_slowapi_response_contract.py with 6 tests including a control needle proving that the installed slowapi still raises, and a DRIFT DETECTOR that fails when any decorated endpoint appears without the headers parameter. §1.1 (revert of 44f3bbe) -> 3 tests failed with the real 500 body; restore by reverting to identical. Description corrected: the incompatibility framing was wrong.
BOB-129	Bug	Fixed (→ Fixed.md)	Medium	
BOB-131	Bug	Queued	Medium	

ATM ID	Type	Status	Severity	Description
BOB-135	Bug	Queued	Low	qbittorrent-proxy podman common crash pre-existing, surfaced during BOB-129 investigation
				Test isolation: test <code>list_hooks_after_create</code> fails in bulk suite (Permission denied / config)
BOB-136	Task	Ready for testing	High	WHAT. A single 2026-08-20 sweep found FOUR workable-item rows whose track state contradicted reality: BOB-076 (closed by commit 99a486e on 2026-08-15, whose message literally says “closes BOB-076” still Queued), BOB-117 (fix landed AND covered by a permanent guard assertion still Queued), BOB-091 (implemented under d1a8479 / a3b16bc / 2a0b543, still Queued), and BOB-008 whose Evidence field still quoted a diagnostic string that BOB-117 removed from source.
				CORRECTION 2026-08-20 (11.4.6). An earlier revision of this item claimed the enabling defect was that “workable-items diff is blind to body_md drift”. That was WRONG and is corrected here rather than quietly edited. The engine is NOT blind. The FLAGLESS INVOCATION is: <code>workable-items diff -db docs/workable_items.db - “diff: DB and Markdown are in sync” (opens ZERO .md files) workable-items -db docs/workable_items.db -issues docs/Issues.md -fixed docs/Fixed.md -> “~ BOB-008 body differs (md=1346 bytes db=1333 bytes)” -> “diff: 1 difference(s) Both measured on the same seeded body only drift. An independent agent proved the mechanism with strace: the flagless form never opens any Markdown file. So this is a 11.4.201(6) FALSE-NULL of the CALLER, not a gap in the comparison logic. Importantly, pre_build_verification invariant 17 already passes both flags, the STANDING GATE was never blind - the drift above was not caused by a blind gate. THE REAL DEFECTS, restated honestly. (1) The flagless form is a false null trap: it reports a confident, reassuring “in sync” while checking nothing. Any caller using it is a green-reporting blind check. It should either default to the conventional Issues.md/Fixed.md paths REFUSE with a clear error, never return</code>

ATM ID	Type	Status	Severity	Description
				clean verdict for a comparison it did not perform. (2) Work commonly lands under commit messages that do not carry the ATM id (GA-14/15/16 for BOB-091), so nothing mechanically links a commit back to its row, and closure depends on a human remembering. 11.4.227: an anchor done state is its SEAM landing, not its TEXT landing. ACCEPTANCE. (a) The flagless invocation no longer returns a clean verdict without reading Markdown, with paired 1.1 mutation proving it. (b) A repository wide grep confirms no caller uses the flagless form. (c) A sweep that flags any non-terminal item whose id appears in a merged commit message, so done-but-closed rows are found mechanically rather than by a human noticing. (d) Honest bounds this does not claim to prevent all drift. RELATED. scripts/hooks/docs-sync-commit-seam.sh (added under BOB-087) now enforces doc/DB sync at the commit stage and passes the flags explicitly, plus an independent body_md re-parse oracle. EVIDENCE. docs/qa/BOB-117/closure-evidence.md, docs/qa/BOB-076/closure-evidence.md, docs/qa/BOB-091/closure-evidence.md. Reported-Via: §11.4.202 reporting directive bug on 2026-08-20T14:46:52Z Reported-By: Claude What (the report verbatim): The merge service on port 7187 wedges after hours of uptime: the port stays bound and the container keeps reporting “healthy”, but every HTTP request hangs until the client times out. Measured 2026-08-20 16:41 UTC+2 on container up 3h53m: curl http://localhost:7186/ -> HTTP 200 in 0.096s (proxy, same process) curl http://localhost:7187/ -> HTTP 000 after 6.0s (merge service, WEDGED) Both ports are served by the SAME process (pid 2330069, fd 3 = 7186, fd 7 = 7187), so this is not a crashed worker - one loop inside a live process has stopped servicing requests while another in the same process is firing. Thread state at the time of the wedge (cat /proc/2330069/task, 16 threads): - tid 2330529: state R, wchan 0 <- ONE thread
BOB-137	Bug	Queued	High	

ATM ID	Type	Status	Severity	Description
				spinning on CPU - tid 2330528: state S, wchan do_sys_poll <- the healthy 7186 loop - the other 14: state S, wchan futex_wait_queue That is the GIL-starvation signature: a thread busy-looping in Python holds the GIL and every other thread queues on the GIL futex. 7186 survives because do_sys_poll releases the GIL; the 7187 async loop needs sustained GIL time to service a request and starves. Process CPU was 20.6% while idle. Corroborating evidence: seven sockets held by the process sit in CLOSE-WAIT with unread bytes still in the receive queue (Recv-Q 162, 162, 162, 79, 1, 1, 1) - clients sent request and hung up, and the app never read it or closed the fd. The listener has also accumulated an unaccepted backlog (Recv-Q 6 on the 7187 LISTEN socket) at first observation. The container log's last line is 2h before the observation, so the service processed nothing in that window. NOT fd exhaustion: only 48 of 16384 fds were open. ROOT CAUSE NOT ESTABLISHED. All four while True loops in the service (routes.py:166, search.py:161, streaming.py:161, streaming.py:359) are correctly bounded and awaited, so the service is not a naive unslept loop. py-spy could not attach to name the spinning frame because the host has kernel.yama.ptrace_scope=0 which denies non-child attach. Per the §11.4.102 Iron Law no fix is proposed until the spinning frame is identified. Next diagnostic step: obtain a Python stack. Either run py-spy INSIDE the container (musl wheel), or set ptrace_scope=0 for the duration of one dump, or add a SIGQUIT/faulthandler.register() dump handler to main.py so the running service can be asked for its own stacks without ptrace. DISCOVERY CHANNEL (§11.4.238): found only by an agent probing the host by hand while investigating an unrelated test-suite regression - NOT by automated QA. This is a cover-up, an escape in its own right; see the sibling regression on the health check that was structurally incapable of observing it. Affected scope: file-scope manifest: download-proxy/src/main.py, download-proxy/src/

ATM ID	Type	Status	Severity	Description
				merge_service/, download-proxy/src/ap streaming.py, docker-compose.yml (qbittorrent-proxy) Reproduction / context: Leave the qbittorrent-proxy container running for several hours with search traffic (a full tests/security run is sufficient). Then: curl -max-time 6 http: localhost:7186/ returns 200; curl -max- time 6 http://localhost:7187/ returns 00 Confirm with: ss -ltnp grep 7187 (unaccepted backlog) and awk '{print \$ /proc/<pid>/task/*/stat (one R thread, n futex_wait_queue). **Acceptance criteri The spinning frame is identified from a Python stack dump (not inferred), the b loop is fixed at its root, and a regressio guard proves 7187 still answers after a sustained-traffic soak. Evidence: before after curl timings on both ports plus a thread-state census showing no permanently-R thread. BOB-141 Ta Queued Low **Reported-Via:** §11.4 reporting directive `task` on 2026-08-20T14:56:38Z **Reported-By:* Claude **What (the report, verbatim):** CLAUDE.md's Architecture section stat "qbittorrent-proxy-go (Go/Gin, opt-in vi profile go) -- replaces the Python proxy 7186, 7187, 7188" The container cannot deliver that. Read from source rather th prose: qBitTorrent-go/Dockerfile:16 CM ["/app/qbittorrent-proxy"] (ONE binary) qBitTorrent-go/Dockerfile:15 EXPOSE 7 7188 (declares 2) cmd/qbittorrent-prox main.go r.Run(fmt.Sprintf(":%d", cfg.ServerPort)) (binds 1) internal/confi config.go:58 ServerPort = MERGE_SERVICE_PORT (default 7187) the qbittorrent-proxy-go container bind 7187 ONLY. webui-bridge is a SEPARAT binary (cmd/webui-bridge, /bridge/heap cfg.BridgePort) that this container neve starts, and nothing in it binds 7186 eith -- although the compose service does se PROXY_PORT=7186 and BRIDGE_PORT=7188 in its environmen which reinforces the wrong impression EXPOSE likewise declares a port nothin binds. WHY THIS MATTERS BEYOND TIDINESS: this prose was used as the

ATM ID	Type	Status	Severity	Description
				source of truth when first authoring container- served_ports.yaml, producing a manifest entry of [7186, 7187, 7188] for that service. The healthcheck gate built on it then FAILED a service whose healthcheck was already correct -- a §11.4.201(1) false positive refusal caused directly by trusting the doc over the Dockerfile. The manifest was corrected against source; the doc was not, and will mislead the next reader the same way. Either the doc is wrong, or the Go container is under-provisioned relative to intent (it should also run webui-bridge and a 7186 listener). Determining WHICH is part of this task -- do not simply rewrite the doc to match the current binary if that intended design was a three-port container. Related: the Go service's compose block sets PROXY_PORT and BRIDGE_PORT to 7186, no process in the container consumes, and EXPOSE lists 7188 unbound. Whichever way the above resolves, those should agree with reality afterwards. **Affected scope: file-scope manifest:** CLAUDE.md (Architecture + Port Map), docker-compose.yml (qbittorrent-proxy-go env, EXPOSE), qBitTorrent-go/Dockerfile **Reproduction / context:** Read qBitTorrent-go/Dockerfile:16 (single CMD against CLAUDE.md's 'replaces the Python proxy on 7186, 7187, 7188'; confirm cfg.ServerPort resolves to MERGE_SERVICE_PORT=7187 in inter/config/config.go:58. **Acceptance criteria:** Doc and container agree, with the direction of the fix decided deliberately (correct the doc, or provision the container to match the documented intent). PROXY_PORT/BRIDGE_PORT env and EXPOSE lines agree with what the container actually binds. BOB-143 1 Queued Medium **Reported-Via:** §11.4.202 reporting directive `bug` on 2026-08-20T15:08:18Z **Reported-By:** Claude **What (the report, verbatim):** `.worktrees/ci-split-workflows/` (13M) and `.worktrees/completion-initiative-phase` (33M) are ORPHANED: `git worktree list` reports only the main checkout, so neither is a registered worktree. Each contains

ATM ID	Type	Status	Severity	Description
				`.git` POINTER FILE whose target gitd no longer exists, so git cannot resolve HEAD, branch, or status inside them -- every query returns empty. They are gitignored (.gitignore:122), so nothing tracks them and nothing will ever notice them drifting. WHY THIS IS NOT COSMETIC: they pollute the scan scope whole-tree gates and manufacture false findings. Measured 2026-08-20 by the §11.4.32 sweep: - `cm_test_mock_pid_explicit_int` reports violations at tests/unit/merge_service/test_deadline_tunable.py:44 -- BOTH in these orphaned trees. The MAIN tree's copy of that file is already hardened (it `mock.pid = 12345` and patches os.kill os.getpgid) and passes the gate cleanly. The finding read as a live §11.4.263 / BOB-126-class defect and was not one. of the 57 "missing anchor carrier" files flagged by the propagation gates were likewise `.worktrees/**`. That is the §11.4.201(1) false-positive shape source from scan scope, and it costs real investigation time: a reader triaging "2 BOB-126 violations" reasonably treats it as a host-safety emergency. CLARIFICATION (established during triage, so the record is not alarming): these trees are NOT a kill(-1) vector. Their `download-proxy/search.py` contains ZEL `os.killpg` calls -- they predate that cleanup code entirely -- so there is no unguarded signal call to reach. Additionally `pyproject.toml` sets `testpaths = ["tests"]`, so a plain `pytest` run does not collect from `.worktrees/`. No host-safety risk was found. The defect is scan-scope noise plus 46M of unreferenced disk. WORK REMOVAL IS NOT DONE AUTONOMOUSLY (§11.4.122 / §11.4.123 §11.4.101): because git cannot resolve their HEAD, it is NOT possible to prove their contents are merged into main. Deleting unprovable-provenance work is exactly the irreversible, operator-owned decision §11.4.122 reserves. Two options for the operator: (a) confirm removal (these are stale dev scratch dirs) -- reversible

ATM ID	Type	Status	Severity	Description
				from backup, so a §9.2 pre-op backup should precede it; or (b) keep them and add <code>.worktrees/</code> to the gate scan-scope exclusion list as a §11.4.224(E)-fenced, checked-in, justified entry. Either way the exclusion list is the cheaper immediate mitigation and does not destroy anything. Affected scope / file-scope manifest: <code>.worktrees/ci-split-workflows/</code>, <code>.worktrees/completion-initiative-phase-0/</code>, gate scan-scope context. Reproduction / context: git worktree list shows only the main checkout; git -C <code>.worktrees/<dir></code> log -1 returns empty (gitdir target missing). Run the §11.4.3 sweep and observe <code>cm_test_mock_pid_explicit_int</code> report 2 violations, both under <code>.worktrees/</code>, while the main-tree file passes the same gate. Acceptance criteria: Whole-tree gate no longer report findings sourced from orphaned worktrees: either the dirs are removed after operator confirmation with §9.2 pre-op backup, or <code>.worktrees/</code> is added to a checked-in §11.4.224(E)-fenced exclusion list with justification. Verify by re-running the sweep and confirming zero <code>.worktrees</code>-sourced findings. BOB-144 Bug Fixed (→ Fixed.md) L Reported-Via: §11.4.202 reporting directive <code>`bug`</code> on 2026-08-20T15:53:5 Reported-By: Claude What (the report, verbatim): <code>`/theme/stream`</code> in <code>download-proxy/src/api/routes.py:167</code> on the disconnect probe with NO guard at <code>while True: if await request.is_disconnected(): break</code> If that probe raises, the generator dies with an UNCAUGHT exception. IMPORTANT — this is NOT the BOB-139 fail-open. The effect here is fail-CLOSED: the stream stops, and the enclosing <code>`finally: store.unsubscribe(queue)`</code> still runs, so subscriber queue is released and nothing leaks. The outcome is CORRECT; the manner is not. What is wrong with it: - terminates via an uncaught traceback rather than a clean SSE <code>`close`</code> event, so the client sees a truncated stream instead of a reason; - the failure is not logged at

ATM ID	Type	Status	Severity	Description
				probe failure, so a systematically raising probe would show up as recurring tracebacks with no diagnosis; - it is inconsistent with the two SSE generators in streaming.py, which after BOB-139 emit `event: close` with reason `disconnect_probe_failed` and log a warning. Three call sites of the same probe now behave two different ways. The BOB-139 fix deliberately did not touch the file (ownership boundary, §11.4.119), and flagged it honestly rather than fixing it out of scope. Acceptance: /theme/stream uses the same probe-failure discipline as the streaming.py generators — a clean close event with the `disconnect_probe_failed` reason plus a logged warning — proven in BOTH directions (§11.4.201(1)): a raising probe closes the stream cleanly AND a normally-connected client still streams uninterrupted. Prefer reusing the shared helper introduced by BOB-139 rather than a third copy of the logic (§11.4.251). Honest boundary (§11.4.6): the product probe-failure rate is UNKNOWN — nobody has measured how often `is_disconnected()` actually raises. This was filed on the inconsistency and the missing diagnosis, not on a measured incident rate. **Affected scope / file-scope manifest:** download-proxy/src/api/routes.py (~line 167, stream_theme) **Reproduction / context:** Monkeypatch Request.is_disconnected to raise, open theme/stream, observe the generator die with an uncaught traceback rather than emitting event: close with reason disconnect_probe_failed as streaming.py now does. **Acceptance criteria:** Same probe-failure discipline as streaming.py (clean close + disconnect_probe_failed reason + logged warning), verified both directions, reusing the BOB-139 helper rather than a third copy. --- ## CLOSED 2026-08-21. /theme/stream's bare `await request.is_disconnected()` now routes through the shared BOB-139 helper and emits a NAMED close reason instead of letting the exception escape. The outcome was already safe (subscriber_count 0->

ATM ID	Type	Status	Severity	Description
				proved the finally released the queue) b safety was a side effect of an escaping traceback, not a decision — one upstre try/except from becoming fail-open. RE failed / 2 passed -> GREEN 6 passed; th pre-fix passes are the deliberate do-not weaken guards and are green in BOTH states. Revert -> same 4 failures -> res byte-identical (sha256 94107969... / 6492c8cf...). Honest boundary: the live container still has the pre-fix module imported, so no live-container runtime verification is claimed. BOB-145 Bu Fixed (→ Fixed.md) High **Reported Via:** §11.4.202 reporting directive `bu on 2026-08-20T16:01:43Z **Reported-B Claude **What (the report, verbatim):** BOB-137 established the root cause: Deduplicator.merge_results() is called a PLAIN SYNCHRONOUS CALL at merge_service/search.py:914 and there executes ON the asyncio event-loop thr While it runs, uvicorn's only loop runs r callback — no accept, no read, no write so port 7187 stops answering entirely. Measured episodes of 9m37s and 5m56 self-clearing, recurring under ordinary traffic. This item is the FIX. BOB-137 is diagnosis and stays separate per the §11.4.102 Iron Law — the investigation deliberately applied no fix. Three candi directions, not yet chosen: (a) OFFLOA hand merge_results to a thread execut (asyncio.to_thread / run_in_executor). Smallest change, immediately unblocks loop. Does NOT make the work cheaper a large enough merge still burns a core and it introduces concurrency around self._last_merged_results and metadata which must be checked for races before is called safe. (b) MEMOISE — _normalize_name is called at FOUR site per comparison, each running 5 re.sub, and _extract_identity_from_result twice more with a ~15-re.search chain. lru_ca has ZERO matches in that module toda the seed's own normalisation is recomputed for every candidate. Caching the per-result normalisation is a large constant-factor win with no concurr

ATM ID	Type	Status	Severity	Description
				risk. (c) REDUCE THE COMPARISON S — the $O(N^2)$ shape itself (blocking/ bucketing by a cheap key before pairwise comparison). Largest win, largest change, highest risk of altering dedup RESULTS which would need its own correctness evidence, not just a speed measurement (b) then (a) is the likely order: (b) is risk-free and may alone bring the merge under the threshold, and (a) guarantees the loop is never blocked regardless. MANDATORY for whoever takes this: - RED FIRST (§11.4.43/§11.4.224): a test that FAILS the current code by demonstrating the loop is blocked during a merge — e.g. assert concurrent request to 7187 is served within a bounded time while a large merge runs. A pure speed benchmark is NOT the RED test; the defect is loop-blocking, not slowness. - BOTH POLARITIES (§11.4.201(1)): the loop stays responsive under a large merge AND dedup results are unchanged for the existing corpus. fix that speeds up merging while changing which duplicates are detected is a different defect. - Use the existing instrument: scripts/diagnostics/bob137_soak.sh reproduced the wedge 22/26; it is the natural GREEN check. - Honest bounds carried from BOB-137: measured growth SUPER-LINEAR but not fully quadratic $N \leq 400$ (2.4-3.4x per doubling vs 4.0). Worst case is quadratic BY CODE STRUCTURE. Do not cite "measured N" **Affected scope / file-scope manifest:** download-proxy/src/merge_service/search.py:914, download-proxy/src/merge_service/deduplicator.py **Reproduction / context:** bash script diagnostics/bob137_soak.sh — reproduced the wedge in 22/26 probes (7187 dead, 7186 alive throughout). **Acceptance criteria:** Port 7187 answers within a bounded time while a large merge runs (loop never blocked), AND dedup results are unchanged for the existing corpus. Proven with the soak repro flipping from 22/26-dead to 0/N-dead, plus a dedup-equivalence check. --- ## CLOSED 2026-08-21. The assumed $O(N^2)$

ATM ID	Type	Status	Severity	Description
				comparison cost was NOT the problem: cProfile at N=400 showed Levenshtein 0.223s (2.3%) while identity extraction, normalisation and 2,107,721 re_comp calls dominated — over only 400 distinct names. Fixed by memoising the pure per string derivations and hoisting them out of the inner loop; comparison count UNTOUCHED, so no matching rule changed. N=400 loop-block 3970ms -> median 349ms; N=800 7337ms -> ~815ms. RED observed at 3970ms vs a 900ms ceiling; revert -> 8994ms; restored byte-identical. Bucketing was IMPLEMENTED and MEASURED before rejection: 0.0% prune rate at N=400 and N=800, and the effective form is unsound because Tier 2 matches on infohash independent of name. Honest limit: the loop still stops while merge_results runs; the window shrank, it did not vanish; removing it needs an await at the search.py call site. BOB-146 Bug Fixed (→ Fixed.md) High **Reported Via:** §11.4.202 reporting directive `bu on 2026-08-20T16:10:50Z **Reported-By: Claude **What (the report, verbatim):** The constitution's §11.4.252 detector (constitution/scripts/gates/cm_dangerous_combination_fail_closed UNDERCOUNTS by 29%. Measured against an independent AST instrument (structure, not text — a valid control needle per §11.4.201(7)) on boba's plug tree: gate reports : 30 AST ground truth 42 missed : 12 false positives : 0 A gate that misses 12 of 42 while reporting a confident number is a §11.4.201(6) false null: its output reads as a census when a FLOOR. Anyone fencing an exclusion against that number (§11.4.224(E)) would write the fence against an undercount and lock the invisible sites out of scope permanently. FOUR DISTINCT CAUSES each independently falsifiable: L1 — TRAILING COMMENT DEFEATS THE REGEX (10 of the 12). The pattern anchors the handler line with `...:[[:space:]]*\$`, soexcept Exception: # noqa: S110never matches. The irony is load-bearing: t

ATM ID	Type	Status	Severity	Description
				sites a human consciously reviewed and annotated are exactly the ones the ga cannot see. L2 – TUPLE CLAUSE DEFEATS REGEX (2 of the 12). The exception-type group is[A-Za-z_.]+, which cannot match soexcept (OSError, ValueError):is invisible. L3 – A COMMENT BETWEENexceptANDpassDEFEATS THE BODY CHECK. The detector reads exactlylineno+1. Zero current instance but demonstrated live. This one has a nasty second-order effect: a well-intentioned reviewer adding an explanatory comment INSIDE a handler may that site vanish from the gate. The triage agent hit this itself – its first patch put comments inside two handler and the resulting count would have read while only 6 sites were genuinely eliminated. It caught and corrected that rather than reporting the better number which is exactly the \$11.4 discipline working. L4 – THE SHAPE ITSELF, not the regex. The body must be exactlypass, soexcept: return